

Yerel RAG Yapay Zeka Asistanı

Microsoft Foundry Local ile Proje Günlüğü

Microsoft Yaz Eğitimi — 15 Günlük Proje Planı

Proje Özeti

Bu proje, Microsoft Foundry Local kullanılarak geliştirilen, tamamen internetsiz (offline) çalışan bir yerel RAG (Retrieval-Augmented Generation) Soru-Cevap asistanıdır. Sistem, bir PDF belgesini (test için: SOS113U — Davranış Bilimleri I, Ünite 1: Sosyolojiye Giriş ders notu) okuyup anlamlı parçalara ayırır, bu parçaları embedding'e çevirip SQLite veritabanında saklar. Kullanıcı bir soru sorduğunda, sistem soruya anlamca en yakın belge parçalarını bulur (retrieval) ve bu parçaları, yerel olarak çalışan bir büyük dil modeline (Foundry Local üzerinden phi-4-mini) göndererek doğal, akıcı bir cevap üretilmesini sağlar. Sistem, belgede olmayan konularda "Bilmiyorum" diyebilecek şekilde tasarlanmıştır, böylece uydurma (hallucination) riski azaltılmıştır.

Proje, 15 günlük bir program dahilinde üç fazda geliştirilmiştir: Faz 1 (Temeller — RAG, embedding, SQLite, Foundry Local kavramlarının öğrenilmesi), Faz 2 (İnşa — belge işleme, arama sistemi ve model entegrasyonunun kodlanması) ve Faz 3 (Test ve Dokümantasyon). Kaynak kod GitHub üzerinde (RabiaKilic-07/rag-project) versiyon kontrolü ile takip edilmiştir.

GÜN 1 — Kurulum ve Temel Kavramlar

Bugünün Hedefi

Foundry Local'i kurmak, bir yapay zeka modelini yerel olarak (internetsiz) çalıştırmak ve RAG'in temel mantığını kavramak.

Öğrenilen Kavramlar

- RAG (Retrieval-Augmented Generation): Modelin, kendi ezberinden değil, verilen bir belgeye bakarak cevap üretmesi. "Açık kitap sınavı" mantığı.
- Fine-tuning ile RAG farkı: Fine-tuning modelin kendi parametrelerini ("beynini") yeniden eğitir; RAG modele dokunmadan, soruyla birlikte ilgili bilgiyi ("context") sağlar.
- Model: Yapay zekanın kendisi — milyarlarca parametreden oluşan, bilgisayara indirilebilen bir dosya.
- Foundry Local: Microsoft'un, yapay zeka modellerini bulut yerine kullanıcının kendi bilgisayarında (offline) çalıştırmasını sağlayan araç. CLI (komut satırı aracı), arka planda çalışan bir servis ve bir model kataloğundan oluşuyor.

Yapılan İşlemler

1. PowerShell yönetici olarak açıldı.

```
winget install Microsoft.FoundryLocal
```

Kullanım şartları onayı (Y) verildi, kurulum tamamlandı.

2. Kurulum doğrulandı:

```
foundry --version
```

3. Mevcut modeller listelendi:

```
foundry model list
```

İlk denemede "No models were returned from the Azure Foundry catalog" hatası alındı. Bu, Foundry Local'in bilinen (GitHub'da kayıtlı) bir sorunu olarak tespit edildi.

4. Çözüm — servis yeniden başlatıldı:

```
foundry service restart
```

Bu komuttan sonra model listesi başarıyla geldi.

5. Küçük bir model (qwen2.5-0.5b, ~0.5 GB) indirildi ve çalıştırıldı:

```
foundry model run qwen2.5-0.5b
```

Model başarıyla indirildi, bilgisayarın GPU'sunda (qwen2.5-0.5b-instruct-trtrtx-gpu:2 varyantı) otomatik olarak yüklendi.

Test Sonucu

Modelle interaktif sohbet ekranında hem İngilizce hem Türkçe test yapıldı:

- "Hello, who are you?" → Model kendini Qwen (Alibaba Cloud) olarak tanıttı.
- "Merhaba, sen kimsin?" → Model Türkçe cevap verdi ("Merhaba! Size nasıl yardımcı olabilirim?"), ancak kimlik bilgisini İngilizce'deki kadar net vermedi — küçük modellerde (0.5B parametre) beklenen bir durum.

Önemli not: Bu aşamada RAG sistemi henüz kurulmadı. Model, sorulara kendi eğitim verisindeki (ezberindeki) bilgiyle cevap verdi. Belge okuma / arama (retrieval) özelliği Gün 4 ve sonrasında eklenecek.

Karşılaşılan Sorunlar ve Çözümleri

Sorun	Neden Oldu	Çözüm
"No models were returned from the Azure Foundry catalog" hatası	Foundry Local'in Azure kataloğuna bağlantı sorunu (bilinen bir hata)	foundry service restart komutu ile servis yeniden başlatıldı, sorun çözüldü

Gün 1 Sonucu

Bilgisayarda internetsiz çalışan, GPU hızlandırmalı bir yapay zeka modeli başarıyla kuruldu ve test edildi. Bu model, ileride kurulacak RAG sisteminin "cevap üreten motoru" olarak kullanılacak.

Bugünün Hedefi

Embedding kavramını gerçek bir Python scripti ile test etmek; metinlerin anlam benzerliğinin sayısal olarak nasıl ölçüldüğünü görmek.

Öğrenilen Kavramlar

- Embedding: Bir metni (kelime, cümle, paragraf) sayılardan oluşan bir vektöre çevirme işlemi. Anlamca yakın metinlerin vektörleri de birbirine yakın oluyor.
- Vektör uzayı ve yakınlık: İki embedding arasındaki anlam benzerliği "cosine similarity" (kosinüs benzerliği) ile ölçülüyor. Sonuç -1 ile 1 arasında bir sayı; 1'e yakın = çok benzer, 0'a yakın = alakasız.
- Chunking (parçalama): Uzun bir belgeyi embedding'e çevirmeden önce küçük parçalara (ör. 200-500 kelime) bölme işlemi. Amaç: embedding'in çok genel olmasını önlemek ve modelin bağlam penceresi sınırına uymak.
- Vektör veritabanının rolü: Çok sayıda embedding arasında hızlı karşılaştırma yapabilmek için (projede SQLite kullanılacak).
- Kütüphane (library) ile model arasındaki fark: pip install ile kod/araçlar (kütüphane) kuruluyor; modelin kendisi (ağırlıklar) ise kod ilk çalıştığında ayrıca internetten iniyor.

Yapılan İşlemler

1. Python kurulu olduğu doğrulandı:

```
python --version
```

2. Proje klasörü oluşturuldu ve içine girildi:

```
mkdir C:\rag-projesi cd C:\rag-projesi
```

3. Embedding kütüphanesi kuruldu:

```
pip install sentence-transformers
```

4. Not Defteri ile test dosyası oluşturuldu:

```
notepad embedding_test.py
```

Bu dosyaya, çok dilli (Türkçe destekli) bir embedding modeli (paraphrase-multilingual-MiniLM-L12-v2) kullanan bir test scripti yazıldı. Script; "kedi", "kedicik", "araba" kelimelerini ve iki farklı izin cümlesini embedding'e çevirip aralarındaki benzerliği hesaplıyor.

5. Script çalıştırıldı:

```
python embedding_test.py
```

İlk çalıştırmada embedding modeli (birkaç yüz MB) otomatik olarak internetten indirildi, ardından sonuçlar başarıyla üretildi.

Test Sonucu

Karşılaştırma	Benzerlik Skoru (cosine similarity)
---------------	-------------------------------------

kedı ↔ kedicik	0.77 (yüksek — anlamca yakın)
kedı ↔ araba	0.33 (düşük — alakasız)
"Yıllık izin süresi 14 gündür." ↔ "Kaç gün izin hakkım var?"	0.53 (orta-yüksek — ortak kelime yok ama aynı konu)

Önemli gözlem: İzin cümlesi ile izin sorusu arasında neredeyse hiç ortak kelime olmamasına rağmen ("yıllık, süresi, 14, gündür" vs. "kaç, hakkım, var") benzerlik skoru yüksek çıktı. Bu, embedding'in kelime eşleşmesi değil, anlamsal (semantic) benzerlik yakaladığını kanıtlıyor — RAG sisteminin belge içinde doğru bilgiyi bulabilmesinin temel sebebi budur.

Vektör Arama Testi (Gün 2'nin Devamı)

İlk testte sadece ikili (1'e 1) karşılaştırma yapılmıştı. Gerçek RAG sisteminde ihtiyaç duyulan şey, bir soruyu birden fazla belge parçasıyla aynı anda karşılaştırıp en alakalı olanı otomatik bulmaktır. Bunu test etmek için yeni bir script yazıldı:

```
notepad vektor_arama_test.py
```

Script, 5 farklı "sahte belge cümlesi" (izin, çalışma saatleri, uzaktan çalışma, öğle molası, deneme süresi hakkında) tanımlayıp bunların embedding'lerini önceden hesapladı. Ardından "Kaç gün izin hakkım var?" sorusunun embedding'i, tüm parçalarla karşılaştırılıp en yüksek benzerlik skoruna sahip parça otomatik olarak seçildi:

```
python vektor_arama_test.py
```

Skor	Belge Parçası
0.531	Yıllık izin süresi 14 gündür. ← en alakalı (doğru seçim)
0.478	Uzaktan çalışma haftada 2 gün ile sınırlıdır.
0.410	Yeni çalışanlar için deneme süresi 2 aydır.
0.383	Şirket çalışma saatleri 09:00 - 18:00 arasındadır.
0.318	Öğle yemeği molası 1 saattir.

Sistem, soruyla en alakalı cümleyi (izin süresiyle ilgili olanı) 5 parça arasından doğru şekilde bulup en üste çıkardı. Bu, RAG sisteminin "retrieval" (arama) mekanizmasının küçük ölçekli, çalışan bir örneğidir. Projede bu mantık büyütülüp SQLite veritabanıyla birleştirilecek: yüzlerce belge parçası arasından, kullanıcı sorusuna en yakın olan(lar) otomatik olarak bulunacak ve Foundry Local modeline gönderilecek.

Gün 2 Sonucu

Embedding kavramı ve vektör arama mantığı, gerçek Türkçe örneklerle uçtan uca test edilerek doğrulandı. Anlamca yakın metinlerin embedding vektörlerinin birbirine yakın çıktığı ve bir sorunun birden fazla belge parçası arasından doğru şekilde eşleştirilebildiği gözlemlendi. Bu adım, RAG sisteminin "arama" (retrieval) kısmının temelini oluşturuyor; bir sonraki adımda bu embedding'ler SQLite veritabanına kaydedilecek.

Bugünün Hedefi

Embedding'leri geçici olarak bellekte tutmak yerine, SQLite veritabanına kalıcı olarak kaydetmeyi öğrenmek; ayrıca RAG sisteminde modele doğru şekilde soru sormak için kullanılacak prompt şablonunun mantığını kavramak.

Öğrenilen Kavramlar

- SQLite: Sunucu gerektirmeyen, tek bir dosyada çalışan hafif veritabanı sistemi. Python'da hazır gelir (sqlite3 modülü), ekstra kurulum gerekmez.
- Tablo / sütun / satır: Veritabanındaki verinin Excel benzeri yapı taşları — tablo bir sayfa, sütun bir başlık, satır ise tek bir kayıt.
- SQL (CRUD): Veritabanıyla konuşmak için kullanılan dil. Temel işlemler: CREATE (oluştur), SELECT/READ (oku), UPDATE (güncelle), DELETE (sil).
- Cursor: Python kodu ile veritabanı arasında komut taşıyan aracı nesne (sqlite3 modülünün otomatik sağladığı bir özellik, ayrıca kurulum gerekmez).
- Prompt engineering: Modelden en doğru/tutarlı cevabı almak için girdi metnini (prompt) dikkatli kurgulama sanatı.
- RAG prompt şablonunun bileşenleri: (1) Talimat — modele görevini tanımlar, (2) Kısıtlama — "bilgi yoksa bilmiyorum de" diyerek uydurma (hallucination) riskini azaltır, (3) Bağlam — retrieval ile bulunan gerçek belge parçası, (4) Soru — kullanıcının sorusu, (5) Cevap tetikleyicisi — modele cevaba başlama sinyali verir.

Yapılan İşlemler

1. Yeni test dosyası oluşturuldu:

```
notepad sqlite_test.py
```

Script şu adımları içeriyor: embedding modelini yükleme, belgeler.db adında bir SQLite veritabanına bağlanma, 'belgeler' adında bir tablo oluşturma (id, metin, embedding sütunlarıyla), 5 örnek belge parçasının embedding'ini hesaplayıp veritabanına INSERT ile kaydetme, son olarak SELECT ile kayıtları okuyup ekrana yazdırma.

2. Script çalıştırıldı:

```
python sqlite_test.py
```

Test Sonucu

5 belge parçası başarıyla veritabanına kaydedildi ve tekrar okunarak doğrulandı:

```
id: 1 - metin: Yıllık izin süresi 14 gündür. id: 2 - metin: Şirket çalışma saatleri 09:00 - 18:00 arasındadır. id: 3 - metin: Uzaktan çalışma haftada 2 gün ile sınırlıdır. id: 4 - metin: Öğle yemeği molası 1 saattir. id: 5 - metin: Yeni çalışanlar için deneme süresi 2 aydır.
```

Bu sonuçla, embedding'lerin artık script kapansa/bilgisayar yeniden başlasa bile kalıcı olarak (belgeler.db dosyasında) saklandığı doğrulandı. Bu, Gün 2'deki "sadece bellekte tutma" durumundan önemli bir ilerlemedir.

Gün 3 Sonucu

SQLite ile kalıcı veri saklama mantığı öğrenilip pratik edildi; embedding'ler artık bir veritabanı dosyasında güvenle duruyor. Ayrıca RAG sisteminde modele nasıl doğru şekilde soru sorulacağı (prompt şablonu ve her bileşeninin işlevi) kavramsal olarak öğrenildi. Bir sonraki adımda (Gün 4, Faz 2 başlangıcı) gerçek bir belgeden (PDF) metin çıkarma ve otomatik parçalama işlemine geçilecek.

GÜN 4-5 — Belge Yükleme (Document Ingestion) — Faz 2 Başlangıcı

Bugünün Hedefi

Faz 2'ye (Build Project) geçildi. Hedef: gerçek bir PDF belgesini okuyup içindeki metni çıkarmak ve bu metni, embedding'e çevrilmeye hazır, anlamlı parçalara bölmek. Ortam olarak Notepad yerine Antigravity (VS Code tabanlı bir kod editörü) kullanılmaya başlandı.

Öğrenilen Kavramlar

- Document ingestion: Bir belgenin diske okunup, metninin çıkarılıp, işlenebilir parçalara bölünmesi sürecinin tamamı.
- Karakter bazlı parçalama: Metni sabit sayıda karaktere (ör. 500) göre bölmek. Basit ama kelime/cümleleri ortadan kesme riski taşır.
- Cümle bazlı parçalama: Metni önce cümlelere ayırıp, cümleleri bir araya toplayarak hedef boyuta yaklaşan parçalar oluşturmak. Kelime/cümle bütünlüğünü korur, daha kaliteli embedding üretir.
- Regular expression (re modülü): Metin içinde belirli kalıpları (ör. cümle sonu noktalama işaretlerini) bulmak için kullanılan arama dili.

Yapılan İşlemler

1. PDF okuma kütüphanesi kuruldu:

```
pip install pypdf
```

2. Antigravity editöründe C:\rag-projesi klasörü açıldı, yeni dosya oluşturuldu:

```
pdf_isle.py
```

3. PDF'ten metin çıkarma kodu yazıldı (PdfReader ile sayfa sayfa metin okunup birleştirildi). Test için 3 sayfalık bir ders notu ("SOS113U — Davranış Bilimleri I, Ünite 1: Sosyolojiye Giriş") kullanıldı.

4. İlk parçalama denemesi — karakter bazlı (500 karakterlik sabit dilimler):

Sonuç: 28 parça elde edildi, ancak bazı kelimeler parçalar arasında ortadan bölündüğü gözlemlendi (örnek: "bir takım" ifadesi "bir" ve "takım" olarak iki ayrı parçaya düştü).

5. Geliştirilmiş yöntem — cümle bazlı parçalama: Metin önce regular expression ile cümlelere ayrıldı, sonra cümleler 500 karakter hedefine yaklaşıncaya kadar bir araya toplanarak parçalar oluşturuldu.

Sonuç: 31 parça elde edildi, hiçbir cümle/kelime ortadan bölünmedi — her parça tam ve anlamlı cümlelerden oluşuyor.

Test Sonucu

Yöntem	Parça Sayısı	Kelime/Cümle Bütünlüğü
Karakter bazlı (500 kr.)	28	Bozuk (kelimeler bölünüyor)
Cümle bazlı	31	Korunuyor (tam cümleler)

PDF'ten çıkarılan Türkçe metnin (özel karakterler dahil: ş, ı, ö, ğ, ü, ç) bozulmadan doğru şekilde işlendiği doğrulandı.

Gün 4-5 Sonucu

RAG sisteminin "belge yükleme" (ingestion) katmanı başarıyla kuruldu: gerçek bir PDF, metne çevrilip anlamlı, cümle bütünlüğü korunan parçalara bölünüyor. Bir sonraki adımda bu parçalar embedding'e çevrilip SQLite veritabanına kaydedilecek — böylece PDF'in tamamı, arama yapılabilir hale getirilmiş olacak.

GÜN 6-7 — *Embedding Oluşturma ve SQLite'a Kaydetme (Gerçek Veriyle) + GitHub Kurulumu*

Bugünün Hedefi

PDF'ten çıkarılan 31 cümle-bazlı parçayı embedding'e çevirip SQLite veritabanına kalıcı olarak kaydetmek — Gün 2 ve Gün 3'te sahte verilerle öğrenilen yöntemi gerçek belge verisine uygulamak. Ayrıca projeyi GitHub'a bağlamak.

Öğrenilen Kavramlar

- Toplu (batch) encoding: `model.encode()` fonksiyonuna cümleleri tek tek değil, bir liste halinde topluca vermek. Çok sayıda parça olduğunda tek tek işlemekten çok daha hızlı çalışır.
- Veritabanını temizleme: `DELETE FROM tablo_adi` komutu, tablonun yapısını korurken içindeki tüm satırları siler. Eski/test verilerini temizleyip gerçek veriyle yeniden doldurmak için kullanılır.
- SQL komutlarının çalıştığı yer: SQL komutları (`DELETE`, `SELECT vb.`) doğrudan terminale/PowerShell'e yazılmaz; Python kodu içinde `cursor.execute("...")` şeklinde bir string olarak çalıştırılır.
- `zip()` fonksiyonu: İki listeyi (parçalar ve embedding'ler) eş zamanlı olarak dolaşp, doğru elemanları birbiriyle eşleştirmeyi sağlayan Python fonksiyonu.
- Kod sırasının önemi: Python dosyayı yukarıdan aşağı çalıştırır; bir değişken (ör. `yeni_parcalar`), kullanılmadan önce tanımlanmış olmalıdır, aksi halde `NameError` hatası alınır.
- Git / GitHub temel akışı: `git init` (depoyu başlat, tek seferlik), `git config` (kimlik tanımlama, tek seferlik), `git remote add origin` (uzak depoya bağlama, tek seferlik). Bundan sonraki her güncellemede sadece `git add` → `git commit -m "..."` → `git push` yeterli.
- `.gitignore` dosyası: Git'e hangi dosyaların GitHub'a yüklenmeyeceğini söyleyen dosya (projede `belgeler.db` bu şekilde hariç tutuldu).

Yapılan İşlemler

1. GitHub reposu oluşturuldu (rag-project) ve proje klasörü bağlandı:

```
git init git config --global user.email "..." git config --global user.name "Rabia Kılıç" git add . git commit -m "İlk commit: PDF okuma ve parçalama kodu" git remote add origin https://github.com/RabiaKilic-07/rag-project.git git push -u origin main
```

İlk push başarıyla tamamlandı, tüm proje dosyaları (.gitignore, PDF, .py dosyaları) GitHub'da görüntülenebilir hale geldi. belgeler.db, .gitignore sayesinde GitHub'a yüklenmedi.

2. pdf_isle.py dosyası, tüm süreci (PDF okuma → parçalama → embedding → SQLite kaydetme) tek bir akışta birleştirecek şekilde yeniden düzenlendi. Karşılaşılan bir NameError hatası (kod sırasının karışması sonucu), dosyanın doğru sırayla yeniden yazılmasıyla çözüldü.

3. Script çalıştırıldı:

```
python pdf_isle.py
```

Test Sonucu

```
Toplam sayfa sayısı: 3 Toplam karakter sayısı: 13978 Toplam parça sayısı: 31 31 parça, embedding'e çevrilip veritabanına kaydedildi. Veritabanındaki toplam kayıt sayısı: 31
```

31 parçanın tamamı başarıyla embedding'e çevrildi ve SQLite veritabanına kaydedildi; eski test verileri (Gün 3'teki 5 sahte cümle) DELETE komutuyla temizlendiği için veritabanında sadece gerçek PDF verisi kaldı.

Karşılaşılan Sorunlar ve Çözümleri

Sorun	Neden Oldu	Çözüm
Git commit sırasında "Author identity unknown" hatası	Bilgisayarda Git kullanıcı kimliği (e-posta/isim) hiç tanımlanmamıştı	git config --global user.email / user.name komutlarıyla kimlik tanımlandı
"DELETE FROM belgeler" komutu PowerShell'e yazıldığında tanınmadı	SQL komutu, Python kodu dışında doğrudan terminale yazılmıştı	Komutun cursor.execute("DELETE FROM belgeler") şeklinde Python kodunun içine yazılması gerektiği netleştirildi
"NameError: name 'yeni_parcalar' is not defined" hatası	Yeni kod eklenirken önceki PDF okuma/parçalama kodu yanlışlıkla silinmiş/üzerine yazılmıştı	Dosyanın tamamı, doğru sırayla (PDF okuma → parçalama → embedding → kaydetme) yeniden yazıldı

Gün 6-7 Sonucu

Gerçek bir PDF belgesi, uçtan uca işlenerek (okuma → parçalama → embedding → SQLite'a kaydetme) tamamen çalışan bir veri katmanına dönüştürüldü. Proje ayrıca GitHub'a bağlandı, versiyon takibi başladı. Bir sonraki adımda (Gün 8-9), kullanıcının soracağı sorulara göre bu 31 parça arasından en alakalı olan(lar)ı otomatik bulan retrieval (arama) pipeline'ı yazılacak.

GÜN 8-9 — Retrieval Pipeline (Arama Sistemi)

Bugünün Hedefi

Kullanıcının sorduğu bir soruya göre, SQLite veritabanındaki 31 parça arasından en alakalı olan(lar)ı otomatik olarak bulan bir arama (retrieval) sistemi yazmak.

Öğrenilen Kavramlar

- BLOB'dan embedding'e geri dönüş: `np.frombuffer(blob, dtype=np.float32)` fonksiyonu, veritabanında ham bayt (BLOB) olarak saklanan embedding'i tekrar kullanılabilir sayı listesine çevirir. `tobytes()` işleminin tam tersidir ("zarfı açma" işlemi).
- Top-k seçimi: Sadece en iyi tek sonucu değil, en iyi k (örn. 3) sonucu seçme yöntemi. Bazı durumlarda doğru cevap birden fazla parçanın birleşiminde olabileceği için tercih edilir.
- `np.argsort()`: Bir listedeki değerleri küçükten büyüğe sıralayıp, değerlerin kendisi yerine sıradaki konumlarını (indekslerini) döndüren fonksiyon. Başına eksi (-) işareti konarak büyükten küçüğe sıralama elde edilir.
- `enumerate()`: Bir listeyi dolaşırken, elemanın kendisiyle birlikte kaçınıcı sırada olduğunu da veren Python fonksiyonu.
- Sabit (hardcoded) soru ile interaktif soru farkı: `soru = "..."` şeklinde kodun içine sabit yazmak yalnızca test amaçlıdır; gerçek kullanımda `input()` fonksiyonuyla kullanıcıdan soru alınacak (Gün 10-11'de eklenecek).

Yapılan İşlemler

1. Yeni bir dosya oluşturuldu (PDF işleme kodundan ayrı tutuldu, çünkü farklı bir görevi var):

```
retrieval_test.py
```

2. Script şu adımları içeriyor: SQLite'dan tüm kayıtları (metin + embedding BLOB) okuma, her BLOB'u `np.frombuffer` ile tekrar embedding'e çevirme, sabit bir test sorusunu embedding'e çevirme, `cos_sim` ile soru embedding'ini 31 parçanın embedding'iyle karşılaştırma, `np.argsort` ile en yüksek skorlu 3 parçayı (top-3) bulup ekrana yazdırma.

3. Script çalıştırıldı:

```
python retrieval_test.py
```

Test Sonucu

Test sorusu: "Sosyoloji nedir?" — sistem, 31 parça arasından en alakalı 3 parçayı doğru şekilde buldu:

Sıra	Bulunan Parça (özet)
1	" Sosyoloji Nedir? Sosyoloji sosyal ilişkiler üzerinde odaklanan... " — doğrudan tanım cümlesi
2 (0.762)	"Sosyolojinin Diğer Sosyal Bilimlerle İlişkisi..."
3 (0.744)	"Her birey toplumsallaşmanın ürünüdür..."

Sistem, en doğru tanımları içeren parçayı 1. sıraya yerleştirdi; 2. ve 3. sıradaki parçalar da konuyla anlamca ilişkili çıktı. Bu, retrieval sisteminin gerçek bir PDF üzerinde isabetli çalıştığını doğruluyor.

Gün 8-9 Sonucu

RAG sisteminin "arama" (retrieval) katmanı başarıyla tamamlandı: kullanıcı sorusuna göre, SQLite'daki gerçek PDF verisinden en alakalı parçalar otomatik olarak bulunabiliyor. Bir sonraki adımda (Gün 10-

11), bulunan bu parçalar Foundry Local modeline gönderilecek, model bunlara dayanarak cevap üretecek ve sistem interaktif hale (kullanıcıdan input() ile soru alan bir döngü) getirilecek.

GÜN 10-11 — Foundry Local Entegrasyonu ve İnteraktif RAG Asistanı

Bugünün Hedefi

Retrieval sisteminin bulduğu belge parçalarını Foundry Local modeline gönderip, modelin bu bilgiye dayanarak doğal bir cevap üretmesini sağlamak; sistemi, kullanıcıdan sürekli soru alabilen interaktif bir döngüye dönüştürmek. Bu adımla RAG sisteminin tüm parçaları (ingestion, embedding, retrieval, generation) ilk kez uçtan uca birleştirildi.

Öğrenilen Kavramlar

- Foundry Local Python SDK (foundry-local-sdk): Python kodundan doğrudan Foundry Local modeline bağlanmayı sağlayan resmi kütüphane. Kullanım şekli sürüm güncellemeleriyle değişebiliyor; güncel sürümde Configuration ve FoundryLocalManager.initialize() ile başlatılıyor.
- Streaming (parça parça) cevap: complete_streaming_chat() fonksiyonu, modelin cevabını tek seferde değil, üretildikçe parça parça (chunk chunk) döndürür — ChatGPT'nin kelime kelime yazması gibi anlık görüntü sağlar.
- while True + break ile interaktif döngü: Program, kullanıcı özel bir kelime ("çıkış") yazana kadar sürekli soru sormaya devam eder; break komutu döngüyü sonlandırır.
- Model boyutunun cevap kalitesine etkisi: Çok küçük modeller (0.5B parametre) tutarsız, tekrarlayan ve talimatlara uymayan (hallucination yapan) cevaplar üretebilirken, daha büyük ve talimat-takibi için optimize edilmiş modeller (phi-4-mini gibi) çok daha güvenilir sonuç verebiliyor.
- Model değiştirmenin kolaylığı: Foundry Local mimarisinde, kullanılan modeli değiştirmek sadece tek bir satırı (model ismini) güncellemek anlamına geliyor; sistemin geri kalanı (retrieval, prompt, döngü) hiç değişmiyor.

Yapılan İşlemler

1. Foundry Local Python SDK kuruldu:

```
pip install foundry-local-sdk openai
```

2. Bağlantı testi için ayrı bir dosya oluşturuldu (foundry_baglanti_test.py). İlk denemede import hatası (ModuleNotFoundError), ardından SDK'nın güncel sürümünde API değişikliği nedeniyle AttributeError alındı. Resmi Microsoft dokümantasyonu incelenerek doğru kullanım şekli (Configuration + FoundryLocalManager.initialize()) uygulandı ve bağlantı başarıyla kuruldu.

3. Tüm sistemi birleştiren ana dosya oluşturuldu:

```
rag_asistan.py
```

Bu dosya sırasıyla: embedding modelini ve Foundry Local'i başlatıyor, SQLite'daki tüm parçaları belleğe okuyor, kullanıcıdan soru alan bir döngü içinde retrieval yapıp en alakalı 3 parçayı buluyor, Gün 3'teki prompt şablonunu doldurup Foundry Local modeline gönderiyor ve cevabı ekrana akıtıyor (streaming).

4. İlk test qwen2.5-0.5b modeliyle yapıldı — cevaplar tutarsız çıktı (aynı cümleler döngüye girip tekrarlandı) ve model, bağlamda olmayan bir soruya ("elma tohumları yenir mi") "Bilmiyorum" demesi gerekirken uydurma bir cevap üretti.

5. Model phi-4-mini olarak değiştirildi (tek satırlık değişiklik):

```
llm_model = manager.catalog.get_model("phi-4-mini")
```

Model tekrar indirilip test edildi.

Test Sonucu

Soru	qwen2.5-0.5b Sonucu	phi-4-mini Sonucu
"Sosyoloji nedir?" (PDF'te var)	Tutarsız, aynı cümleler sonsuz döngüde tekrarlandı	Akıcı, tutarlı ve PDF içeriğine dayalı doğru cevap
"Elma tohumu/çekirdeği yenir mi?" (PDF'te YOK)	Uydurma, alakasız bir cevap üretti (hallucination)	"Bilmiyorum. Verilen bağlam bu konuyu içermez." — doğru ve güvenilir

phi-4-mini modeli, hem cevap tutarlılığı hem de prompt talimatına uyma (bilmediğinde "Bilmiyorum" deme) konusunda qwen2.5-0.5b'ye göre belirgin şekilde daha güvenilir sonuçlar verdi.

Gün 10-11 Sonucu

Proje, PDF okuma → parçalama → embedding → SQLite kaydetme → retrieval → Foundry Local ile cevap üretme zincirinin tamamının uçtan uca çalıştığı, interaktif bir RAG asistanına dönüştü (rag_asistan.py). Sistem, hem bildiği konularda doğru/akıcı cevap verebiliyor hem de bilmediği konularda uydurma yapmadan bunu belirtebiliyor. Bu, Faz 2'nin (Build Project) fiilen tamamlandığı anlamına geliyor. Sıradaki adım Faz 3: fonksiyonel testler, ek belge desteği (opsiyonel) ve kapsamlı dokümantasyon/demo hazırlığı.

GÜN 12 — Fonksiyonel Testler — Faz 3 Başlangıcı

Bugünün Hedefi

Faz 3'e (Test & Wrap-up) geçildi. Hedef: rag_asistan.py'yi farklı soru kategorileriyle sistematik olarak test etmek, sistemin doğruluğunu, dürüstlüğü (uydurma yapmama) ve sağlamlığını (çökmeme) değerlendirmek.

Öğrenilen Kavramlar

- RAG sistemlerinde test kategorileri: (a) Doğrudan sorular — belgede açıkça olan bilgi, (b) Yeniden ifade edilmiş sorular — farklı kelimelerle sorulan aynı anlam, (c) Kapsam dışı sorular — belgede olmayan bilgi, sistemin "Bilmiyorum" diyebilme yeteneğini test eder, (d) Sınır durumlar (edge cases) — boş girdi, çok kısa/belirsiz soru gibi alışılmadık durumlar.
- Değerlendirme kriterleri: Doğruluk, alaka (retrieval isabeti), dürüstlük (hallucination yapmama), akıcılık, hız.

- continue komutu: Bir döngüde, o anki adımın geri kalanını atlayıp döngünün başına dönmeyi sağlayan Python komutu; geçersiz/boş girdilerde işlemi (retrieval + model çağrısı) hiç başlatmadan kullanıcıya tekrar soru sordurmak için kullanılır.

Yapılan İşlemler

rag_asistan.py, python rag_asistan.py komutuyla çalıştırılıp aşağıdaki test soruları sırayla denendi.

Test Sonucu

Kategori	Soru	Sonuç	Değerlendirme
Doğrudan soru	"Sosyoloji nedir?"	Doğru, akıcı, PDF'e dayalı cevap	Başarılı
Yeniden ifade	"Sosyoloji ne ile ilgilenir?"	Farklı kelimelerle sorulsa da doğru cevap bulundu	Başarılı — embedding'in anlamsal eşleştirme gücünü kanıtladı
Kapsam dışı	"Elma çekirdeği yenir mi?"	"Bilmiyorum" dedi, uydurmadı	Başarılı
Sınır durum (belirsiz)	"ne"	"Bilmiyorum" dedi, çökmedi	Başarılı
Sınır durum (boş girdi)	(boş, sadece Enter)	İlk denemede alakasız/uydurma bir cevap üretti	Hata bulundu → koda boş girdi kontrolü eklenerek düzeltildi

Bulunan Hata ve Düzeltmesi

Boş bir soru (sadece Enter'a basılması) gönderildiğinde, sistemin anlamsız bir embedding üretilip alakasız bir belge parçasını "bağlam" olarak model modele gönderdiği ve modelin bu alakasız bağlam hakkında konuşarak kafa karıştırıcı bir cevap ürettiği tespit edildi. Çözüm için rag_asistan.py'deki döngüye bir kontrol eklendi:

```
if soru.strip() == "":    print("Lütfen bir soru yazın.\n")    continue
```

Bu değişiklikten sonra boş girdi gönderildiğinde sistem, retrieval veya model çağrısı yapmadan doğrudan "Lütfen bir soru yazın." mesajını veriyor ve tekrar soru bekliyor. Test tekrarlanarak düzeltmenin çalıştığı doğrulandı.

Gün 12 Sonucu

rag_asistan.py, 5 farklı test senaryosuyla (doğrudan, yeniden ifade, kapsam dışı, iki farklı sınır durum) sistematik olarak değerlendirildi. 4 senaryo ilk denemede başarılı geçti; 1 senaryoda (boş girdi) bulunan hata anında düzeltildi ve doğrulandı. Sistem artık daha sağlam ve kullanıcı dostu. Bir sonraki adımda (Gün 13), proje boyunca tutulan bu günlük gözden geçirilip kapsamlı dokümantasyon tamamlanacak.

GÜN 13 — Kapsamlı Dokümantasyonun Tamamlanması

Bugünün Hedefi

Gün 1'den Gün 12'ye kadar tutulan günlüğü tamamlayıcı üç bölümle desteklemek: mimari özeti, kullanım kılavuzu ve genel sonuç/değerlendirme. Ayrıca GitHub reposu için ayrı bir README.md dosyası hazırlamak.

Mimari Özeti

Sistem 5 ana bileşenden oluşur:

- Belge İşleme (pdf_isle.py): PDF'i okur, metni çıkarır, cümle bütünlüğünü koruyarak parçalara (chunk) böler, her parçayı embedding'e çevirip SQLite veritabanına (belgeler.db) kaydeder.
- Arama / Retrieval (retrieval_test.py mantığı, rag_asistan.py içinde entegre): Kullanıcı sorusunu embedding'e çevirir, veritabanındaki tüm parça embedding'leriyle karşılaştırıp (cosine similarity) en alakalı top-k parçayı bulur.
- Prompt Oluşturma: Bulunan parçaları ve kullanıcı sorusunu, modelin bağlama sadık kalmasını ve bilmediğinde "Bilmiyorum" demesini sağlayan bir şablona yerleştirir.
- Model / Generation (Foundry Local + phi-4-mini): Prompt'u yerel olarak çalışan dil modeline gönderir, modelin ürettiği cevabı akış (streaming) halinde kullanıcıya gösterir.
- Kullanıcı Arayüzü: Terminal tabanlı, sürekli soru sormaya izin veren interaktif bir döngü (rag_asistan.py).

Veri akışı özetle: PDF → metin → parçalar → embedding → SQLite (bir kez, önceden) ⇒ soru → embedding → SQLite'dan en yakın parçalar → prompt → Foundry Local → cevap (her sorguda).

Kullanım Kılavuzu

Projeyi çalıştırmak için gerekli adımlar:

1. Foundry Local kurulumu: winget install Microsoft.FoundryLocal (Windows).
2. Python kütüphanelerinin kurulumu: pip install pypdf sentence-transformers foundry-local-sdk openai
3. Belgenin işlenmesi (bir kez yapılır, belge değişmediği sürece tekrar gerekmez): python pdf_isle.py
4. RAG asistanının başlatılması: python rag_asistan.py
5. Terminalde "Sorunuz:" istemine soru yazılır; "çıkış" yazılarak program sonlandırılır.

Sonuç ve Değerlendirme

Proje, hedeflenen "internetsiz çalışan, belge tabanlı Soru-Cevap asistanı" hedefine ulaşmıştır. Sistem, Gün 12'deki testlerde hem bilgi içeren sorularda doğru/akıcı cevaplar üretebildiğini hem de kapsam dışı sorularda uydurma yapmadan "Bilmiyorum" diyebildiğini göstermiştir.

Öğrenilen en önemli teknik dersler: (1) embedding tabanlı arama, kelime eşleşmesi olmasa dahi anlamca doğru sonuçlar bulabiliyor; (2) model boyutu, RAG sisteminin cevap kalitesini ciddi şekilde etkiliyor — küçük modeller (0.5B) tutarsız ve hallucination yapmaya daha yatkın, orta boy talimat-odaklı modeller (phi-4-mini) çok daha güvenilir; (3) basit ama sistematik testler (özellikle sınır durumlar), teslimden önce fark edilmesi zor hataları (boş girdi sorunu gibi) ortaya çıkarabiliyor.

Bilinen sınırlamalar: Sistem şu an tek bir PDF ile çalışacak şekilde kurulmuştur (birden fazla belge desteği teknik olarak mümkün ama henüz eklenmemiştir); arayüz terminal tabanlıdır, grafik/web arayüzü yoktur (bonus olarak eklenmesi düşünülebilir).

